



1. BDD EMPLEADA PARA ADMINISTRAR EL NEGOCIO DE LA EMPRESA

1. BDD EMPLEADA PARA ADMINISTRAR EL NEGOCIO DE LA EMPRESA.....	1
1.1. Carga de datos.....	2
1.1.1. Preparar el ambiente para cargar un archivo BLOB de prueba.....	2
Ejemplo.....	4
1.2. Presentación del proyecto.....	4
1.2.1. Presentación 1: Creación de la BDD.....	4
Ejemplo.....	5
1.2.2. Presentación 2: Carga de datos por copia manual.....	5
Ejemplo.....	5
1.2.3. Presentación 3: Carga de datos con transparencia de distribución.....	6
Ejemplo.....	6
Ejemplo.....	7
1.2.4. Presentación 4: Validación de resultados – Insert y datos replicados.....	10
1.2.5. Presentación 5: Eliminación de datos.....	11
Ejemplo.....	11
1.2.6. Presentación 6: Validación de resultados - delete.....	12

1.1. Carga de datos

En esta última parte se realizará la carga de datos para validar los conceptos de transparencia implementados anteriormente. Para realizar esta actividad se proporciona un archivo llamado `carga-inicial.zip`. Este archivo contiene una carpeta llamada `carga-inicial`. En dicha carpeta se encuentra una serie de scripts SQL con sentencias `insert` así como una muestra de imágenes empleadas para validar el soporte de datos BLOB.

Se recomienda colocar la carpeta `carga-inicial` dentro del mismo directorio donde se encuentran todos los scripts SQL del proyecto. Esta convención permitirá usar las rutas que se muestran en este documento y no se necesitará realizar modificaciones a las rutas o nombres de directorios.

Adicional al archivo zip, se incluyen algunos scripts SQL para realizar la validación de resultados. Copiar estos archivos al directorio donde se encuentran todos los scripts SQL del proyecto.

1.1.1. Preparar el ambiente para cargar un archivo BLOB de prueba

Crear un script llamado `s-07-ilap-configuracion-soporte-blobs.sql`. El archivo deberá ser ejecutado en todos los nodos para configurar y crear los objetos necesarios para poder leer datos BLOB/CLOB del sistema de archivos local. El script deberá realizar lo siguiente:

- Crear un objeto tipo directory llamado `proyecto_final_laptops_dir` que apunte a un directorio del sistema de archivos. Este directorio contendrá archivos PDF de prueba para ser insertados en la tabla `laptop`. Por simplicidad se elige al directorio `/tmp/bdd/proyecto-final/imagenes/laptops`.
- Crear un objeto tipo directory llamado `proyecto_final_facturas_dir` que apunte a un directorio del sistema de archivos. Este directorio contendrá archivos PDF de prueba para ser insertados en la tabla `servicio_laptop`. Por simplicidad se elige al directorio `/tmp/bdd/proyecto-final/imagenes/facturas`.
- Para que el usuario `ilap_bdd` pueda crear, leer y escribir en estos directorios, este deberá contar con el privilegio `create directory`. Se recomienda agregar este privilegio en el script `s-01-ilap-usuario.sql`.
- Crear una función llamada `fx_carga_blob` que se empleará en instrucciones `insert` para leer un archivo binario y guardarlo en una columna BLOB. El código del script se muestra a continuación.

```
--@Autor:          Jorge A. Rodríguez C
--@Fecha creación:  dd/mm/yyyy
--@Descripción:     Script empleado para configurar el
--                  Soporte de datos BLOB.
```

Prompt Creando objetos para leer datos BLOB

Prompt creando directorios

```
-- el usuario ilap_bdd debe tener el privilegio create any directory
```

```
create or replace directory proyecto_final_facturas_dir
  as '/tmp/bdd/proyecto-final/imagenes/facturas';
create or replace directory proyecto_final_laptops_dir
  as '/tmp/bdd/proyecto-final/imagenes/laptops';
```

Prompt creando funcion para leer datos BLOB

```
create or replace function fx_carga_blob(
  v_directory_name      in varchar2,
  v_src_file_name       in varchar2 ) return blob is
  v_src_blob bfile:=bfilename(v_directory_name,v_src_file_name);
  v_dest_blob blob := empty_blob();
  v_src_offset number := 1;
  v_dest_offset number :=1;
  v_src_blob_size number;
begin
  if dbms_lob.fileexists(v_src_blob) =0 then
    raise_application_error(-20001, v_src_file_name
      ||' El archivo no existe ');
  end if;
  --abre el archivo
  if dbms_lob.isopen(v_src_blob) = 0 then
    dbms_lob.open(v_src_blob,dbms_lob.LOB_READONLY);
  end if;
  v_src_blob_size := dbms_lob.getlength(v_src_blob);
  --crea un objeto Lob temporal
  dbms_lob.createtemporary(
    lob_loc => v_dest_blob,
    cache  => true,
    dur    => dbms_lob.call
  );
  --Lee el archivo y escribe en el blob
  dbms_lob.loadblobfromfile(
    dest_lob    => v_dest_blob,
    src_bfile   => v_src_blob,
    amount      => dbms_lob.getlength(v_src_blob),
    dest_offset => v_dest_offset,
    src_offset  => v_src_offset
  );
  --cerrando blob
  dbms_lob.close(v_src_blob);
  if v_src_blob_size <> dbms_lob.getlength(v_dest_blob) then
    raise_application_error(-20104,'Invalid blob size. Expected: '
      ||v_src_blob_size
      ||', actual: '
```

```

        || dbms_lob.getlength(v_dest_blob)
    );
end if;
return v_dest_blob;
end;
/
show errors

```

Crear un script llamado `s-07-ilap-main-soporte-blobs.sql` Este script deberá ejecutar el script anterior en cada nodo.

Ejemplo

```

--@Autor:          Jorge A. Rodríguez C
--@Fecha creación:
--@Descripción:    Script principal empleado para configurar el soporte
--                  de datos BLOB en Los 4 nodos.

Prompt configurando directorios y otorgando registros.
--jrcbdd_s1
Prompt configurando soporte BLOB para jrcbdd_s1
connect ilap_bdd/ilap_bdd@jrcbdd_s1
@s-07-ilap-configuracion-soporte-blobs.sql
--jrcbdd_s2
Prompt configurando soporte BLOB para jrcbdd_s2
connect ilap_bdd/ilap_bdd@jrcbdd_s2
@s-07-ilap-configuracion-soporte-blobs.sql
--arcbdd_s1
Prompt configurando soporte BLOB para arcbdd_s1
connect ilap_bdd/ilap_bdd@arcbdd_s1
@s-07-ilap-configuracion-soporte-blobs.sql
--arcbdd_s2
Prompt configurando soporte BLOB para arcbdd_s2
connect ilap_bdd/ilap_bdd@arcbdd_s2
@s-07-ilap-configuracion-soporte-blobs.sql
Prompt Listo !

```

1.2. Presentación del proyecto

Para realizar la presentación del proyecto, se deberán crear y ejecutar los siguientes scripts.

1.2.1. Presentación 1: Creación de la BDD

Crear un script `s-08-ilap-presentacion-1.sql` encargado de ejecutar todos los scripts que generan la BDD.

Ejemplo

```
--@Autor:          Jorge A. Rodríguez C
--@Fecha creación: dd/mm/yyyy
--@Descripción:     Script encargado de crear la BDD

clear screen
whenever sqlerror exit rollback;
Prompt Iniciando con la creación de la BDD.

@s-01-ilap-main-usuario.sql
@s-02-ilap-ligas.sql
@s-03-ilap-main-ddl.sql
@s-04-ilap-main-sinonimos.sql
@s-05-ilap-main-vistas.sql
@s-06-ilap-main-triggers.sql
@s-07-ilap-main-soporte-blobs.sql

Prompt Listo !
exit
```

1.2.2. Presentación 2: Carga de datos por copia manual

Crear un script llamado `s-08-ilap-presentacion-2.sql` El script se conectará a cada PDB y realizará la inserción de los datos en donde no existe fragmentación, ni esquema de replicación. En este caso, el script contendrá la carga para la tabla `status_laptop`

Ejemplo

```
--@Autor:          Jorge A. Rodríguez C
--@Fecha creación: dd/mm/yyyy
--@Descripción:     Archivo de carga inicial para catálogos replicados.
clear screen
whenever sqlerror exit rollback;
--Para visualizar export NLS_LANG=SPANISH_SPAIN.WE8ISO8859P1

Prompt =====
Prompt Cargando catálogos de forma manual en jrcbdd_s1
Prompt =====
connect ilap_bdd/ilap_bdd@jrcbdd_s1
delete from status_laptop;
```

```
@carga-inicial/status_laptop.sql
commit;

Prompt =====
Prompt Cargando catálogos de forma manual en jrcbdd_s2
Prompt =====
connect ilap_bdd/ilap_bdd@jrcbdd_s2
delete from status_laptop;
@carga-inicial/status_laptop.sql
commit;

Prompt =====
Prompt Cargando catálogos de forma manual en arcbdd_s1
Prompt =====
connect ilap_bdd/ilap_bdd@arcbdd_s1
delete from status_laptop;
@carga-inicial/status_laptop.sql
commit;

Prompt =====
Prompt Cargando catálogos de forma manual en arcbdd_s2
Prompt =====
connect ilap_bdd/ilap_bdd@arcbdd_s2
delete from status_laptop;
@carga-inicial/status_laptop.sql
commit;

Prompt Listo!
exit
```

1.2.3. Presentación 3: Carga de datos con transparencia de distribución

Crear un script llamado `s-08-ilap-presentacion-3.sql`. Este archivo contendrá la carga inicial de todas las tablas globales así como la ejecución de algunas consultas para validar el correcto funcionamiento. Este script a su vez invoca a un script llamado `s-08-ilap-presentacion-3.sh`. El archivo contiene un pequeño programa (Shell script) que realiza la copia de las imágenes en los directorios configurados anteriormente. El código de este script se muestra a continuación. Leer su contenido y en caso de requerir, actualizar rutas.

Ejemplo

Shell Script.

```
#!/bin/bash
#@Autor:          Jorge A. Rodríguez C
#@Fecha creación: dd/mm/yyyy
#@Descripción:     Copia archivos binarios

#Si ocurre un error, el programa termina.
set -e
set -o pipefail

#En caso de no encontrar el directorio, extrae el contenido del archivo zip
if [ ! -d "/tmp/bdd/proyecto-final/imagenes/laptops" ]; then
    echo "Copiando imágenes - laptops de muestra "
    mkdir -p /tmp/bdd/proyecto-final/imagenes
    unzip carga-inicial/laptops.zip -d /tmp/bdd/proyecto-final/imagenes
else
    echo "=> Las imágenes - laptops de muestra ya fueron copiadas"
fi
if [ ! -d "/tmp/bdd/proyecto-final/imagenes/facturas" ]; then
    echo "Copiando imágenes - facturas de muestra"
    mkdir -p /tmp/bdd/proyecto-final/imagenes
    unzip carga-inicial/facturas.zip -d /tmp/bdd/proyecto-final/imagenes
else
    echo "=> Las imágenes - facturas de muestra ya fueron copiadas."
fi
#actualiza permisos
chmod -R 755 /tmp/bdd/proyecto-final
```

Ejemplo

Script SQL

```
--@Autor:          Jorge A. Rodríguez C
--@Fecha creación: dd/mm/yyyy
--@Descripción:     Archivo de carga inicial - fragmentos
clear screen
set serveroutput on
--Para visualizar export NLS_LANG=SPANISH_SPAIN.WE8ISO8859P1

Prompt =====
Prompt Preparando carga de Datos
Prompt =====

Prompt =>Seleccionar la PDB para insertar datos
Prompt => Asegurarse que las imágenes existen en ambos servidores
```

Prompt => Ejecutar s-08-ilap-presentacion-3.sh en ambos antes de continuar

```
connect ilap_bdd/ilap_bdd@&pdb
```

Prompt Personalizando el formato de fechas

```
alter session set nls_date_format='yyyy-mm-dd hh24:mi:ss';
```

Prompt => Al ocurrir un error se saldrá del programa y se hará `rollback`
`whenever sqlerror exit rollback`

Pause => Presionar Enter para Iniciar con la extracción de datos binarios,
Ctrl-C para cancelar

```
--Invoca a un shell script para realizar la extracción y copia de archivos  
!sh s-08-ilap-presentacion-3.sh
```

```
Prompt =====
```

```
Prompt ¿ Listo para Iniciar con la carga ?
```

```
Prompt =====
```

Pause => Presionar Enter para Iniciar, Ctrl-C para cancelar

Prompt => Realizando limpieza inicial

```
set feedback off
```

Prompt Eliminando datos de historico_status_laptop

```
delete from historico_status_laptop;
```

```
--completar. Tener cuidado con el orden de eliminación,  
--debe ser eliminada con base a las dependencias de las tablas.
```

Prompt => Realizando Carga de datos

Prompt cargando tipo_tarjeta_video

```
@carga-inicial/tipo_tarjeta_video.sql
```

Prompt cargando tipo_procesador

```
@carga-inicial/tipo_procesador.sql
```

Prompt cargando tipo_monitor

```
@carga-inicial/tipo_monitor.sql
```

Prompt cargando tipo_almacenamiento

```
@carga-inicial/tipo_almacenamiento.sql
```



```
Prompt cargando sucursal
@carga-inicial/sucursal-1.sql
--es_venta = 1, es_taller = 0
@carga-inicial/sucursal-2.sql
--es_venta= 1, es_taller = 1
@carga-inicial/sucursal-3.sql

Prompt cargando sucursal_taller
-- id 1 al 1000
@carga-inicial/sucursal_taller-1.sql
-- id 2001 al 3000
@carga-inicial/sucursal_taller-2.sql

Prompt cargando sucursal_venta
-- id 1001 al 2000
@carga-inicial/sucursal_venta-1.sql
-- id 2001 al 3000
@carga-inicial/sucursal_venta-2.sql

Prompt cargando laptop (con datos BLOB)
--Laptops sin reemplazo
--@carga-inicial/laptop-1.sql
@carga-inicial/laptop-1-empty-blob.sql

--Algunas de estas laptops tienen reemplazo
--@carga-inicial/laptop-2.sql
@carga-inicial/laptop-2-empty-blob.sql

Prompt cargando laptop_inventario
@carga-inicial/laptop_inventario.sql

Prompt cargando historico_status_laptop
@carga-inicial/historico_status_laptop-1.sql
@carga-inicial/historico_status_laptop-2.sql

Prompt cargando servicio_laptop (con datos BLOB)
--@carga-inicial/servicio_laptop-1.sql
@carga-inicial/servicio_laptop-1-empty-blob.sql

--@carga-inicial/servicio_laptop-2.sql
@carga-inicial/servicio_laptop-2-empty-blob.sql

Prompt Carga de datos completa. Haciendo commit!
commit;
```

exit

Observar las partes en Negritas. Se muestran 2 versiones del script. Uno de ellos contiene el prefijo `empty_blob`. Esto significa que la tabla contiene columnas BLOB y que el valor de la columna está vacía, es decir, se está haciendo uso de la función `empty_blob`. El script que se proporciona es justamente el script que contiene la función `empty_blob`. Lo anterior implica que se deben realizar las siguientes acciones:

- Renombrar el archivo para que coincida con el valor esperado, es decir, remover el prefijo `empty_blob`.
- Editar el archivo para que ya no haga uso de la función `empty_blob`. En su lugar se deberá invocar a la función `fx_carga_blob` que fue creada anteriormente. Esta función será la encargada de leer el contenido de un archivo binario y regresar un objeto BLOB que será insertado en las tablas `laptop` y `servicio_laptop`.
- La función recibe 2 parámetros:
 - El nombre del objeto tipo `DIRECTORY` que apunta al directorio donde se encuentran los archivos binarios. Recordando, anteriormente se crearon 2 objetos `DIRECTORY` llamados `PROYECTO_FINAL_LAPTOPS_DIR` y `PROYECTO_FINAL_FACTURAS_DIR`.
 - El nombre del archivo cuyo contenido será guardado en la BDD. Se deberá seleccionar un nombre de archivo aleatorio del directorio. Los archivos están numerados iniciando en 1. Se cuenta con un total de 40 archivos png de muestra para simular la foto de una laptop, y un total de 40 archivos png que se emplean para simular facturas. Con estas características, es posible crear una cadena que contenga un número aleatorio entre estos rangos. Por ejemplo: `'lap'||num_aleatorio||'.png'` o `'fa'||num_aleatorio||'.pdf'`. Se recomienda emplear la función `dbms_random.value` y `round` para generar un número aleatorio.
- Con estos cambios, cada sentencia `insert` incluirá un objeto BLOB. Se recomienda dejar esta actividad hasta el final para reducir los tiempos de carga de datos.
- Realizar este cambio al menos a 100 registros.
- Para hacer el reemplazo, se recomienda emplear un editor de texto y realizar una operación de "Find & replace".

1.2.4. Presentación 4: Validación de resultados – Insert y datos replicados

Ejecutar el script `s-08-ilap-presentacion-4.plb`. Se recomienda ejecutar este script en cada una de las 4 PDBs para validar que todo esté funcionando correctamente.

1.2.5. Presentación 5: Eliminación de datos

Crear un script `s-08-ilap-presentacion-5.sql`. Este se encargará de verificar que la transparencia de eliminación funciona correctamente. El script deberá eliminar los datos de todas las tablas. Hacer uso de un procedimiento almacenado que invoque

instrucciones delete en orden correcto para realizar el vaciado de los datos. Se crea una transacción distribuida para realizar esta operación. Cualquier error que ocurra durante el borrado de datos provocará que se haga un **rollback** de todo el proceso.

Ejemplo

```
--@Autor:          Jorge A. Rodríguez C
--@Fecha creación: dd/mm/yyyy
--@Descripción:    Script de eliminación de datos

Prompt Seleccionar la PDB para realizar la eliminación de datos

connect netmax_bdd/ilap_bdd@&pdb
set serveroutput on
Prompt Eliminando datos ...
declare
  v_formato varchar2(50) := 'yyy-mm-dd hh24:mi:ss';

begin
  dbms_output.put_line(to_char(sysdate,v_formato)
    || ' Eliminando datos de playlist');
  delete from historico_status_laptop;

--completar

commit;
exception
  when others then
    dbms_output.put_line('Errores detectados al realizar la eliminación');
    dbms_output.put_line('Se hara rollback');
    dbms_output.put_line(dbms_utility.format_error_backtrace);
    rollback;
    raise;
end;
/
Prompt Listo!
exit
```

1.2.6. Presentación 6: Validación de resultados - delete

Ejecutar el script **s-08-ilap-presentacion-6.plb** Se recomienda ejecutar este script en cada una de las 4 PDBs para validar que todo esté funcionando correctamente. No olvidar volver a cargar datos antes de realizar la prueba. El script verifica que la transparencia de distribución para instrucciones delete funciona correctamente.

¡FIN!